USER GUIDE

RadioDroid

Android CHIRP-compatible universal radio programmer

github.com/jnyer27/RadioDroid

CHIRP drivers · USB OTG · Bluetooth LE · Android

RadioDroid User Guide

RadioDroid **Chirp Programmer** is an Android app that programs amateur and GMRS radios using the same Python drivers as the CHIRP desktop application. No PC is required: connect via USB OTG or Bluetooth LE and edit channels directly on your phone or tablet.

Current release: v4.5.0 — See GitHub Releases for APK downloads and release notes. This guide is also published at **jnyer27.github.io/RadioDroid** (same content as the PDF attached to each release).

Recent shipped themes (v4.2–v4.5): RepeaterBook search in a CHIRP-like flow (amateur JSON API, US **GMRS** proximity via HTML with **export** fallback when tone fields are login-gated); **BLE/USB** reconnect hardening (no more stuck “address in use” after disconnect); **Bluetooth off** → prompt to enable before BLE scan; **TSQL** tones visible everywhere (list/editor/import/download); **NICFW TD-H3** and similar paths no longer show a bogus **88.5 Hz** TX PL when RX/TSQL holds the real CTCSS. Older release notes: v4.4.0, v4.5.0.

Table of contents

1. Requirements
2. Connecting to a radio
3. Bluetooth LE adapters (v2.1+)
4. Main screen
5. Downloading and uploading
6. Editing channels
7. Radio settings
8. CHIRP CSV import and export
9. RepeaterBook search (CHIRP-style)
10. Radio backup (JSON)
11. Customize main screen
12. Supported radios
13. Privacy policy

Schematic layouts in this guide

Some sections include **boxed diagrams** that resemble the phone UI (toolbar, lists, dialogs). They are **not screenshots**—wording, colors, and spacing are simplified. Your theme and Android version may differ slightly.

Requirements

- **Android 7.0 or later** (API 24+)
- **USB OTG cable** (for wired programming) **or** a **Bluetooth LE–capable** Android device and a BLE-to-serial adapter (for wireless)
- A **CHIRP-compatible radio** — if it works with CHIRP on a PC, it should work with RadioDroid

---

Connecting to a radio

USB OTG

1. Connect the programming cable to the radio.
2. Connect the other end to your Android device using a USB OTG adapter.
3. Open RadioDroid; the app will detect the USB serial device.
4. Tap the ⋮ menu → **Select Radio Model…** and choose your radio's vendor and model.
5. Tap **Connect** (or **Load from radio** / **Download**) to open the port and read the radio.

Bluetooth LE

1. Pair your BLE-to-serial adapter with your Android device in system Bluetooth settings (if your phone requires it).
2. In RadioDroid, tap **Connect** and choose **Connect via BLE**, then select your adapter from the scan list.
3. Select your radio model (⋮ → **Select Radio Model…** if needed).
4. Tap **Load from radio** to download the channel list.

Schematic: BLE device list

```
┌──────────────────────────────────────┐
│ Nearby devices                       │
├──────────────────────────────────────┤
│ ← Connect via BLE                    │
├──────────────────────────────────────┤
│  HM-10-5C12                          │
│  UART adapter · RSSI -58             │
├──────────────────────────────────────┤
│  NUS-Prog                            │
│  Nordic UART · Paired                │
└──────────────────────────────────────┘
```

Schematic. Device names vary; tap a row to select that adapter.
After a successful download, the main screen shows the channel list and you can edit and upload.

Bluetooth LE adapters (v2.1+)

v2.1 improves support for common low-cost BLE-to-serial dongles used with Baofeng, TIDRADIO, and similar radios.

- **UART services supported** — The app looks for a serial/UART-style GATT service in this order: **HM-10 / TI-style** (`FFE0`), **Nordic UART**, **Microchip / ISSC**, **NICFW / TD-H3** (`FF00`). The first service present on the adapter is used; TX/RX characteristics are picked automatically (notify/indicate for data from radio, write for data to radio).
- **Scan filtering** — BLE scan only shows devices that **advertise** one of those UART services. That reduces clutter but means: if your dongle does **not** list the UART UUID in its advertisement packet, it may **not** appear in the list even though it would work after a direct connect (future app versions may add an optional “show all BLE devices” mode).
- **MTU and chunk size** — Writes default to **20-byte** chunks (safe for all adapters). The app then negotiates a **moderate** MTU; if negotiation fails or never completes, operation continues at 20 bytes so flaky adapters are less likely to disconnect.

If connection fails, try another USB cable path, ensure the dongle is powered and not paired exclusively to another app, and confirm the radio model is correct in **Select Radio Model…**.

v4.4+: If **Bluetooth is turned off**, starting a BLE scan opens a system-style prompt to **enable Bluetooth** first (instead of an empty device list).
v4.3: After **link loss** or switching adapters, reconnect should be more reliable (unique local sockets and proper close-before-open on BLE and USB serial).

Main screen

- **Toolbar**
- **Connect / Disconnect** — open or close the connection to the radio.
- **Load from radio** — download channel memory from the radio (clone/sync in).
- **Save to radio** — upload the current channel list back to the radio (clone/sync out). You'll get a confirmation before overwriting the radio memory.
- **Channel list** — scrollable list of channels (e.g. 1–198). Each row shows:
- **Channel number** and **name** on the first line, with **badges** for transmit **power**, **mode**, **duplex**, and (when they do not duplicate those) the two fields you pick in **Customize main screen**.
- **RX frequency** on the next line (main readout).
- Optional **tone** summary (`T: … · R: …`) when tones are set.
- For radios that support them: a **single compact radio-specific** line (**Memory.extra** — e.g. merged **Groups:**, bandwidth, busy lock). Empty or **None** values are omitted so the list stays scannable.
- **⋮ Menu**
- **Search Channels** — show a search bar to filter by name, group, or frequency.
- **Import CHIRP CSV from File…** — load a `.csv` exported from desktop CHIRP.
- **Import CHIRP CSV from Clipboard…** — paste and import CSV text.
- **Search RepeaterBook…** — CHIRP-style country/state/service UI; fetches data from the official RepeaterBook API and merges selected repeaters into your channel list (see RepeaterBook search).
- **Select Radio Model…** — pick vendor and model for the connected radio.

- **Customize main screen** — choose two extra values shown as **badges** on each channel row (when not redundant with power/mode/duplex/tones).
- **Radio settings…** — open the driver's global settings (backlight, beeps, etc.); only shown when the radio supports it and a memory image is loaded.
- **Save EEPROM dump…** — save the current in-memory image to a file (for backup or inspection).
- **Import Radio Backup…** — load a RadioDroid **JSON** backup (channels, optional EEPROM, settings).
- **Export Radio Backup…** — share/save that JSON backup (see [Radio backup (JSON)]).
- **Export Raw EEPROM…** — binary image for clone radios (tools / low-level backup).
- **Export CHIRP CSV (selected slots)…** — export selected channels as a CHIRP CSV to share or use in desktop CHIRP.

Schematic: main list

```
┌──────────────────────────────────────────┐
│ Connected · USB                          │
├──────────────────────────────────────────┤
│ RadioDroid                            ⋮  |
├──────────────────────────────────────────┤
│ [Disconnect]  [Load from radio]          │
│               [Save to radio]            │
├────┬─────────────────────────────────────┤
│  1 │ GMRS 1           [130] [FM]        │
│    │ 462.5625                           │
│    │ T: 100.0 Hz · R: 67.0 Hz           │
│    │ Groups: A, G · BW: Wide           |
│    │ BusyLock: Off                     |
├────┼─────────────────────────────────────┤
│  2 │ GMRS 2    [Low] [NFM] [+5000kHz]   │
│    │ 462.5875                           │
│    │ Groups: B · BW: Narrow · BusyLock: On |
└────┴─────────────────────────────────────┘
```

Schematic. App bar, connection actions, and sample channel rows: badges for power/mode/duplex (and Customize slots when set), frequency on its own line, optional tone summary, and one dense Memory.extra line (groups merged; empty/None omitted).

Multi-select and bulk actions

- **Enter selection mode** — **Long-press** a channel. A **selection bar** appears at the bottom with the count of selected channels.
- **Add or remove channels** — Tap rows to toggle selection. Tap **Done** to exit selection mode.
- **Search → Select All** — Enable **Search Channels** (⋮), filter the list by name, **group label** (from driver extras), or frequency, then tap **Select All** to select every **non-empty** channel that matches the filter; bulk actions then apply only to those slots.
- **Selection bar actions** (left to right after the count):
- **Move up / Move down** — Nudge selected channels in the list.
- **Move to slot** — Move the selection to another slot range.
- **TX power** — Set the same transmit power on all selected non-empty channels (uses the driver's power level names).
- **Radio-specific field** — Button with the **tag / group** icon. Opens a list of **writable** parameters from the driver's channel-extra schema (the same **Memory.extra** fields as **Radio-specific** in the channel editor). Choose a field, then enter or pick a value; RadioDroid applies it to **each selected non-empty channel**. Read-only fields are omitted. If no schema is available yet, load from the radio or open a channel once so the app can read the driver layout. **This replaces the old "bulk set channel groups (A–O)" control:** group and other radio-specific values now live only in **Memory.extra** when the driver defines them, and you change them per channel in the editor or for many channels at once with this action.
- **Export CSV** — Share the selected channels as CHIRP CSV.
- **Clear** — Empty the selected slots.

Import, export, and radio settings are enabled only when channels are loaded (and, for radio settings, when the radio has settings support and is connected or has a clone image).

Downloading and uploading

- **Download (Load from radio)**
  Reads the full channel memory from the radio using the CHIRP driver's sync/clone logic. **v2.2+:** An **indeterminate** progress bar and **rotating status text** show that work is in progress (the app does not receive a real percent/byte count from the CHIRP bridge). When it finishes, the channel list is filled and you can edit and save to file or upload later.
- **Upload (Save to radio)**
  Writes the current in-memory channel list back to the radio. The app asks for confirmation because this overwrites the radio's memory. Upload uses the same driver as download (e.g. full clone for many handhelds). **v2.2+:** The same indeterminate bar and rotating messages appear during upload.

Schematic: transfer in progress

```
┌──────────────────────────────────────────┐
│ RadioDroid                               │
├──────────────────────────────────────────┤
│ [Load from radio ▒]  [Save to radio ▒]   │
├──────────────────────────────────────────┤
│ ▓▓▓▓▓▓▓▓▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒    |
│ Reading radio memory…                    │
│                                          │
│ (v2.2+: status text rotates; no          │
│  numeric percent from radio bridge)      │
└──────────────────────────────────────────┘
```

Schematic. Indeterminate bar and status line during Load from radio or Save to radio.

For **clone-mode** radios (e.g. many Baofeng, TID Radio, Retevis), the app keeps a full EEPROM image in memory after download. **Radio settings** and channel edits apply to that image; **Save to radio** sends the whole image back.

Editing channels

Tap a channel in the list to open the **channel editor**.

Schematic: channel editor

```
┌──────────────────────────────────────────┐
│ ← Channel 1                              │
├────────────────┬─────────────────────────┤
│ RX frequency   │ 462.5625                │
│ Duplex         │ [Simplex]               │
│ Name           │ GMRS 1                  │
│ Power          │ [High]                  │
│ Mode           │ [FM]                    │
├────────────────┴─────────────────────────┤
│ [Cancel]                        [Done]   │
└──────────────────────────────────────────┘
```

Schematic. Typical fields; your radio may show more options (tones, radio-specific rows, etc.).

- **RX Frequency** — receive frequency in MHz.
- **Duplex / Offset** — Simplex, **+** (positive offset), **−** (negative offset), or **Split** (separate TX frequency). Offset is in kHz or MHz as appropriate.
- **Channel name** — short label (length limit depends on the driver, often 8–16 characters).
- **Power** — transmit power (e.g. High, Low, or driver-specific levels).
- **Mode** — e.g. FM, NFM, AM, USB (driver-dependent).
- **TX tone / RX tone** — CTCSS or DCS encode/decode; choose from the tone list or “Off”.
- **Radio-specific settings** — **Memory.extra** parameters from the CHIRP driver (e.g. bandwidth, group membership, busy lock / BCL when exposed as extras). Shown as spinners, switches, or text fields depending on type. Some bool extras are forced off when duplex uses an offset, if the driver requires it.

Tap **Done** to save the channel and return to the list, or **Cancel** to discard changes.

Radio settings

⋮ → **Radio settings…** opens a dynamic form built from the CHIRP driver’s **global** settings (e.g. backlight, timeouts, beeps, display options).

- The list is grouped by the driver’s setting groups; expand or collapse sections as needed.

- Use the **search** field at the top to filter by setting name or value.
- Change values, then tap **Update settings** to write them to the radio (or to the in-memory image for clone radios; then use **Save to radio** to upload).

Schematic: radio settings

```
┌──────────────────────────────────────────┐
│ ← Radio settings                         │
├──────────────────────────────────────────┤
│  Search settings                         │
│  [ Filter by name or value… ]            │
├──────────────────────────────────────────┤
│  ▼ Display & sound                       │
│     Backlight  │ [On]                    │
│     Beeps      │ [Key only]              │
├──────────────────────────────────────────┤
│          [Update settings]               │
└──────────────────────────────────────────┘
```

Schematic. Group titles and fields come from the CHIRP driver; layout varies by model.
Radio settings are available only when: - The radio driver supports settings, and
- The app has a connection to the radio **or** a clone image already loaded (for clone radios, settings are applied to the in-memory image).

CHIRP CSV import and export

v3.3+: Export uses the same column set as CHIRP's `Memory.CSV_FORMAT` (including **RxDtcsCode** and **CrossMode**). **Mode** values follow desktop CHIRP `MODES` (e.g. **NFM** / **NAM** for narrow FM/AM on supported drivers such as **TD-H3 nicFW 2.5**).

- **Import**
- **From file:** ⋮ → **Import CHIRP CSV from File…** → choose a `.csv` exported from desktop CHIRP. Channels are merged into the current list by slot.
- **From clipboard:** Copy CSV text (e.g. from CHIRP or email), then ⋮ → **Import CHIRP CSV from Clipboard…**. Same merge behavior as file import.
- **Export**

- Select one or more channels (long-press or use the selection UI), then use the export action (e.g. from the menu or toolbar). Name the file and share or save. Format is CHIRP CSV so you can open it in CHIRP on a PC or share with others.

RepeaterBook search (CHIRP-style)

⋮ → **Search RepeaterBook…** opens a screen modeled on desktop CHIRP's RepeaterBook flow. Data comes from **RepeaterBook**'s official export endpoints (`export.php` / `exportROW.php` JSON), with client-side filters similar to CHIRP's `do_fetch`.

Prerequisites

1. **Load a memory image first** — **Load from radio** or **Import Radio Backup…** so the app already has the radio's channel / clone context in memory. Without that, the screen exits with a reminder to load from radio or import a backup.
2. **API access in the APK** — Official builds ship with **RepeaterBook** credentials in `BuildConfig` (token + contact email). If you **build from source** and both are empty, you'll see a configuration toast: set **`REPEATERBOOK_APP_TOKEN`** and **`REPEATERBOOK_CONTACT_EMAIL`** in `local.properties` (see project README / developer notes) and rebuild. **401 Unauthorized** usually means RepeaterBook rejected the **User-Agent** or auth style — the app's error hint lists `REPEATERBOOK_USER_AGENT` and **`REPEATERBOOK_AUTH_MODE`** options (`bearer`, `x_rb_app_token`, etc.) to match what your API approval email specifies.

Searching

- Pick **country** and **state** (or "All" where supported), **Amateur** vs **US GMRS**, then run the search. **GMRS** is only available when the country is **United States**.
- **Proximity (lat / lon / miles)** — Use **Use my location** (may request **location permission**) or type coordinates. US **Amateur** proximity uses RepeaterBook's HTML **Proximity 2.0**-style request; the app then enriches rows with **Uplink / Downlink tone** from repeater **detail** HTML when needed. **US GMRS** proximity uses a GMRS HTML path; if tone cells are hidden behind login, **v4.4+** can fall back to authenticated JSON **`export.php`** (`stype=gmrs`) when your token is valid, so **PL/TSQ** still map into channels.

- **Filters** — Band/mode chips and feature filters behave like the CHIRP source: e.g. for coordinate searches you must pick at least one **band or** enter a frequency (RepeaterBook rule). A quick text filter narrows the result list.

Importing into memory

- Select repeaters (per-row checkboxes; menu **Select all visible** / **Clear**), then **Import**. RadioDroid maps each row to a **CHIRP-style channel** (frequencies, duplex, tones including DCS strings) and merges into the current list by slot, similar in spirit to importing CSV.
- Downstream you can **Save to radio** or **Export Radio Backup…** as usual.

Network and location

Using this feature contacts **repeaterbook.com** (and related pages) over HTTPS. **Coarse/fine location** is optional and only used when you ask for **Use my location** for proximity searches — RadioDroid does not upload your channel list to RepeaterBook.

Radio backup (JSON)

⋮ → **Export Radio Backup…** saves a JSON file with vendor/model, all **channels**, optional `eeprom_base64` (clone radios), and **radio settings** as stored in the app.

- **Default filenames** start with a sanitized `Vendor_Model` prefix (special characters become `_`), then a timestamp — e.g. `Baofeng_UV_5R_radiodroid_backup_20250316_143022.json` for backup and `Baofeng_UV_5R_eeprom_20250316_143022.img` for **Export Raw EEPROM…**.
- **Settings entries** are written as `path` **+** `value` **only** (small files). The app does **not** embed long CHIRP UI lists (e.g. every power-level label) in the backup — when you import, RadioDroid rebuilds field types and options from the selected driver, same as after **Import Radio Backup…**.
- **Import:** ⋮ → **Import Radio Backup…** — select the same **radio model**, then pick the `.json` file. Clone backups with EEPROM restore full fidelity; others load channels and queue settings for the next **Save to radio**.

- **Clone radios:** When `eeprom_base64` is present, RadioDroid loads that image, then applies any `settings` entries from the file into the in-memory image (same mechanism as **Radio Settings** → **Save**), and **then** decodes channels from the result — so `path` / `value`**-only settings** in the backup are not ignored. The app keeps the raw image in sync with the channel list when you export, save to radio, or after bulk edits / CSV import, so the JSON `channels` array and `eeprom_base64` should match after a normal export.

Customize main screen

⋮ → **Customize main screen**

Choose **Slot 1** and **Slot 2** — two values shown as **badges** on each channel row (e.g. bandwidth, duplex, or other driver-defined **Memory.extra** fields). The list already shows **power**, **mode**, and **duplex** as badges; slots are skipped when they would duplicate those or the tone line. This only changes the display; it does not alter channel data.

Supported radios

RadioDroid ships with the same set of drivers as CHIRP. **Any radio that works with CHIRP on a PC should work with RadioDroid** over USB OTG or (with a BLE adapter) over Bluetooth LE. Examples of supported families (this is not a full list):

- **Baofeng** — UV-5R, BF-F8HP, UV-82, UV-K5, and others
- **TID Radio** — Pick the entry that matches your firmware: **TD-H3** is the stock CHIRP layout; **TD-H3 nicFW 2.5** is the separate driver for **nicFW V2.5** codeplugs (different EEPROM layout). Using the wrong one will misread channels or show blanks. Other TID models (e.g. TD-H6, TD-H8, TD-M11) have their own entries.
- **Custom `.py` driver (trust)** — **v3.4.0+:** Tapping **Load .py Driver** shows a reminder that the file runs **inside RadioDroid** with the same kind of access as the rest of the app (including USB/Bluetooth programming). Only use driver files from **sources you trust**; they are not run in a separate locked-down sandbox.

- **Duplicate driver (sideloaded `.py`)** — If you used **Load .py Driver** for a module that is **also bundled** (e.g. `tidradio_h3_nicfw25.py`), the app may try to load it again on startup and hit a **duplicate driver** warning—the built-in copy is the one you should use. Delete the extra copy from `files/custom_drivers/` (e.g. with adb) or clear app data to drop sideloaded drivers; the FAB is only needed for drivers **not** in the build.
- **Retevis** — RA25, RT85, and others
- **Yaesu** — FT-60, FT-65, and others
- **Kenwood** — TH-D74, and others
- **BTECH** — UV-50X3, and others

Select **Select Radio Model…** from the menu to see the full list for your build. For driver-specific behavior (e.g. radio-specific settings or channel extras), the app builds the UI from the driver; see the Dynamic driver-based UI doc in the repository for technical details.

Privacy policy

RadioDroid is **local-first**: channel memories, EEPROM images, and backups stay on your device unless **you** export or share them. The app does **not** include third-party analytics SDKs for programming workflows. **Bluetooth** and **USB** are used to talk to **your radio** (or adapter), not to servers operated by the RadioDroid project.
Network: The app declares `INTERNET` so **RepeaterBook search** can reach **repeaterbook.com** when you open that screen and run a query. Ordinary **programming-only** use (download/save channels without RepeaterBook) does not require you to use that feature. Optional **location** access applies only if you use **Use my location** in RepeaterBook proximity mode.
Full policy (kept in sync with the app architecture):

- **Online (MkDocs):** jnyer27.github.io/RadioDroid/privacy-policy/
- **Source in the repo:** privacypolicy.md

Tips

- **First time:** Connect the radio, select the exact model, then **Load from radio**. After the list is loaded, you can disconnect and still edit; reconnect when you want to **Save to radio** or open **Radio settings**.
- **Clone radios:** For radios that use a full EEPROM clone, **Radio settings** and channel edits apply to the in-memory image. Use **Save to radio** to write everything back in one go.
- **Search:** Use **Search Channels** to quickly find channels by name, group, or frequency.
- **Backup:** Use **Export Radio Backup…** for a portable JSON snapshot (channels + settings + EEPROM when available), or **Save EEPROM dump…** / **Export Raw EEPROM…** for a raw clone image before big changes.
- **RepeaterBook:** Load from radio (or restore backup) first, then ⋮ → **Search RepeaterBook…**; pick repeaters and import, then review tones and **Save to radio** when ready.

RadioDroid uses CHIRP drivers and Chaquopy to run them on Android.

USER GUIDE

RadioDroid

Android CHIRP-compatible universal radio programmer

github.com/jnyer27/RadioDroid

CHIRP drivers · USB OTG · Bluetooth LE · Android

RadioDroid Privacy Policy

RadioDroid Chirp Programmer logo

Effective date: March 22, 2026

App: RadioDroid — Android CHIRP-compatible radio programmer (`com.radiodroid.app`)

Open-source repository: github.com/jnyer27/RadioDroid

This policy describes how RadioDroid handles information based on **how the app is built and behaves today**. RadioDroid is **local-first**: it runs CHIRP radio drivers on your device (via Chaquopy) and talks to your radio over **USB OTG**, **Bluetooth Low Energy**, or **Bluetooth serial** when you choose to connect. It does **not** require a user account and does **not** include third-party analytics or advertising SDKs in the official open-source build.

Summary

Topic	Practice
Servers operated by the app	The app does **not** send your channel lists, EEPROM images, or radio settings to RadioDroid developers. There is **no** `INTERNET` **permission** in the official manifest for routine operation.
Analytics / crash reporting	**Not embedded** in the project as shipped from this repository (no Firebase, Crashlytics, or similar in the app module).

RADIODROID

Where your data lives	Channel data, clone images, backups, and preferences stay **on the device** unless **you** export, share, or back them up (e.g. CSV, JSON, file save, Android share sheet).
Radio connections	Data is exchanged **between your phone/tablet and your radio** (or BLE adapter) over USB/BT — not routed through RadioDroid's servers (there are none).

If you use an **unofficial or modified build**, privacy practices may differ; verify with whoever provided the APK.

Information the app processes (on your device)

RadioDroid may **store and display** on-device:

- **Channel memories** — frequencies, names, tones, modes, power, and driver-specific fields (`Memory.extra`) as supported by the selected CHIRP driver.
- **Radio EEPROM / clone images** — for "clone-mode" radios, a binary image may be held in memory for programming and backup flows.
- **Radio settings** — values exposed by the driver's settings UI (e.g. squelch, display options), when applicable.
- **Preferences** — e.g. last selected radio model/vendor/baud (via Android `SharedPreferences`) so the app can restore your last choice.
- **Optional files you create** — CHIRP CSV exports, JSON backups, EEPROM dumps — only where **you** choose to save or share them.

Processing is performed **locally** to operate the app (edit channels, program the radio, import/export files you select).

Bluetooth and USB

- **Bluetooth (BLE and classic serial):** Used to discover/connect to adapters and transfer programming data to the radio. Pairing, scan, and connection behavior follow **Android's Bluetooth APIs** and the permissions you grant (see the app manifest: Bluetooth

scan/connect; on older APIs, location may be required for BLE scan — **maxSdkVersion 30** for legacy location permissions).

- **USB OTG:** Used to communicate with USB-serial radios or cables attached to the device. Optional auto-launch when a matching USB device is attached is configured in the manifest; you can still connect from within the app.

Identifiers: Bluetooth device **names and MAC addresses** may appear in the UI when you scan or connect. They are used **locally** for connection; they are **not** uploaded to RadioDroid by the official app.

Network / internet

The **official RadioDroid app** from this repository does **not** declare general `INTERNET` permission for core programming features. Networking is **not** used for:

- Syncing channels to a cloud,
- Logging in,
- Telemetry to the developer.

When you might use the internet anyway (outside the app's control):

- **Downloading the APK** or reading documentation from **GitHub** or **GitHub Pages** (`jnyer27.github.io/RadioDroid`) uses your browser or GitHub's infrastructure — that is between you and those services.
- **Sharing a file** (CSV, backup, etc.) via another app may upload content to a destination **you** choose (email, cloud drive, etc.).

Bundled **CHIRP** Python code includes utilities (e.g. URL fetch helpers) used in **desktop CHIRP**; RadioDroid's Android bridge does not turn the app into a general-purpose network client for programming. Features that would require network access are not part of the documented Android UX.

Custom drivers

Advanced users may **load a custom CHIRP driver** from device storage. That code runs **inside the app process** with the same trust model as bundled drivers. **Only load drivers from sources you trust** — malicious code could, in principle, access data visible to the app. The official project does not review third-party drivers you sideload.

Android backup

The application manifest sets `android:allowBackup="true"`. Whether app data is included in **Google/Android backup** depends on your **device backup settings** and OEM behavior. Sensitive radio programming data could be part of device backups if your system backs up app data.

Children’s privacy

RadioDroid is a **technical tool for licensed radio operators and enthusiasts**. It is **not directed at children** under 13 (or the age required in your jurisdiction). The app does not knowingly collect personal information from children.

Changes to this policy

Updates will be published in the **same repository** (`privacypolicy.md` and the online copy under **User guide** → **Privacy policy**). The **effective date** at the top will be revised when material changes are made.

Open source

Source code is available under the project **LICENSE** in the repository. You can inspect permissions, dependencies, and data flows in the Android module and Python bridge.

Contact

For privacy-related questions about the **open-source project**, open an issue on **github.com/jnyer27/RadioDroid**. This project is maintained by contributors; there is no dedicated commercial data controller or DPA process unless separately agreed.

This document is provided for transparency and convenience; it is not legal advice.